FOCUSONFORCE

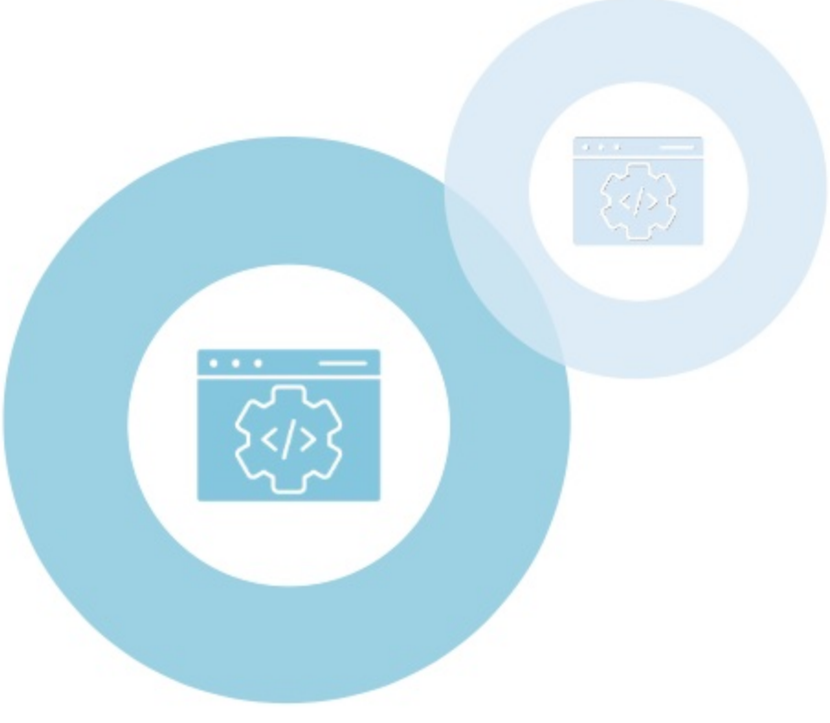
Deploying

Given a scenario, describe the capabilities, limitations, and considerations when using the Metadata and Tooling APIs for deployment.

Table of Contents

After studying this topic, you should be able to:

- Describe the capabilities, considerations, and limitations of Metadata API.
- Describe the capabilities, considerations, and limitations of Tooling API.
- Describe the appropriate use of Metadata or Tooling API and any related considerations for the given business requirements.



Introduction

Salesforce allows the use of **Metadata** and **Tooling APIs** to perform operations that involve metadata. Metadata API can be used to move metadata between orgs during the development process. It supports operations such as **retrieving, deploying, exporting, and deleting metadata**.

Up to 10,000 files can be retrieved or deployed using Metadata API. Moreover, the **maximum file size** of a deployed .zip file is **39 MB**.

Tooling API can be used when **fine-grained access** to an org's metadata is required. It supports the use of **SOQL queries** to retrieve smaller pieces of metadata. **REST resources** and **SOAP calls** can be used to access Tooling API functionality. Certain SOQL and SOSL operations are not supported for the EntityDefinition and FieldDefinition objects.

Metadata & Tooling APIs

Metadata API Capabilities

Metadata API can be used to perform metadata operations, such as moving metadata between organizations.

Metadata API Considerations

Various considerations apply to the use of Metadata API, such as using the destructiveChanges.xml file for deleting components.

Metadata API Limitations

Up to 10,000 files can be retrieved or deployed at once. The maximum file size of a .zip file is 39 MB.

Tooling API Capabilities

Tooling API is used for fine-grained access to an org's metadata. For example, it can be used to retrieve custom field definitions.

Tooling API Considerations

REST resources and SOAP calls can be used to access Tooling API functionality. Error-free save operations can be allowed to complete with warnings.

Tooling API Limitations

Certain SOQL and SOSL operations are not supported for the EntityDefinition and FieldDefinition objects.

Metadata API

Table of Contents

Capabilities of Metadata API

Metadata API is used to **perform metadata operations**, such as **moving metadata between orgs** during the development process.

METADATA OPERATIONS

Metadata API can be used to **deploy, retrieve, create, update, or delete** customization information.

METHODS

Metadata can be moved using **deploy()** and **retrieve()** calls. The other method is using **source push** and **pull commands** that move only **changes in metadata** and use **source tracking**.

CUSTOMIZATIONS & TOOLS

Metadata API can be used to **develop customizations** and **build tools** that manage the metadata model.

Capabilities of Metadata API

Metadata API is used to **perform metadata operations**, such as **moving metadata between orgs** during the development process.

ACCESS

Salesforce Extensions for Visual Studio Code or **Salesforce CLI** can be used to access Metadata API functionality.

TYPES & COMPONENTS

Metadata API works with **metadata types** and **components**. A **metadata type** defines the structure of application metadata. A **metadata component** is an instance of a metadata type.

EXAMPLE

MyCustomObject__c component is an instance of the **CustomObject** metadata type.

Capabilities of Metadata API

Metadata API is used to **perform metadata operations**, such as **moving metadata between orgs** during the development process.

LOCAL FILE SYSTEM

Metadata can be moved from **production to the local file system** for development. It can then be pushed to a **shareable repository**.

SCRATCH ORG

Local changes can be **moved to and from a scratch org**. The scratch org can be used along with the local file system for **development** and **testing**.

INTEGRATION POINTS

During development, metadata can be **moved to sandboxes** for **integrating** changes, testing, and collaborating with the team.

Capabilities of Metadata API

Metadata API is used to **perform metadata operations**, such as **moving metadata between orgs** during the development process.

DEPLOYMENT

Metadata can be **moved from a source control system such as Git into production** in the final step of the development lifecycle.

PRODUCTION-LEVEL CHANGES

Metadata can be **moved for production-level changes**, such as merging or splitting orgs.

LARGE CHANGES

Metadata API is more suitable than other APIs for **deploying large changes** to an org.

Metadata API Considerations

Various considerations apply to the use of **Metadata API calls** to **manage customizations**.

EXPORT

Customizations can be exported as **XML metadata files** using the **retrieve()** call.

MIGRATION & MODIFICATION

Configuration changes can be **migrated between environments** and **existing changes can be modified** using the **deploy()** and **retrieve() calls**.

CRUD-BASED DEVELOPMENT

Customizations can be managed **programmatically** using **CRUD-based metadata calls**.

Metadata API Considerations

Various considerations apply to the use of **Metadata API calls** to **manage customizations**.

ZIP FILE

The **deploy()** and **retrieve()** calls are used to deploy and retrieve a .zip file. The file has a **project manifest (package.xml)** that lists what to retrieve or deploy, and **one or more XML components** organized into folders.

MANIFEST FILE

These elements can be defined in a **package.xml file**:

1) **<fullName>** contains the name of the server-side package.
2) **<types>** contains the metadata type name and the named members.
3) **<members>** contains the fullName of the component.
4) **<name>** contains the metadata type.
5) **<version>** contains the API version number that's used when the .zip file is retrieved or deployed

Metadata API Considerations

Various considerations apply to the use of **Metadata API calls** to **manage customizations**.

DELETION

Components can be deleted with the **deploy()** call using a **destructive changes manifest file** that lists the components to be removed. A deployment can **only delete components** or **delete and add components**.

DESTRUCTIVE CHANGES

Deleting components uses the **same procedure** as **deploying** components, but a delete manifest file named **destructiveChanges.xml** needs to be included. The file lists the **components to delete**.

CUSTOM METADATA TYPES

A **custom metadata type** is defined as a **CustomObject** and has a suffix of **__mdt**. **CustomMetadata** represents a **record** of a custom metadata type.

Metadata API Limitations

Certain **limitations** are applicable to the use of Metadata API.

NUMBER OF FILES

Up to **10,000 files** can be deployed or retrieved at once using the Metadata API. The limit for AppExchange files is 35,000.

FILE SIZE

The **maximum file size** of a deployed or retrieved .zip file is **39 MB**. The size limit is 400 MB if the files are uncompressed in an unzipped folder.

UNSUPPORTED TYPES

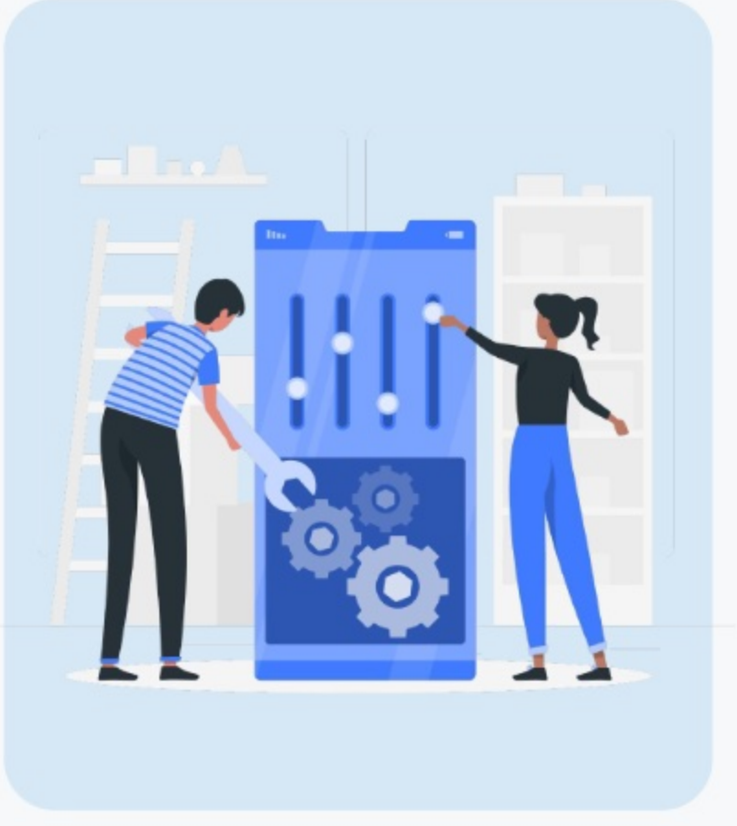
Some metadata types **cannot be deployed or retrieved** with Metadata API. Changes to these types need to be performed **manually**.

Tooling API

Table of Contents

Capabilities of Tooling API

Tooling API can be used for **fine-grained access** to an org's metadata.



SOQL

Tooling API has **SOQL capabilities** that allow **retrieving smaller pieces of metadata**, which improve performance and make it suitable for developing interactive applications.

EXISTING TOOLS

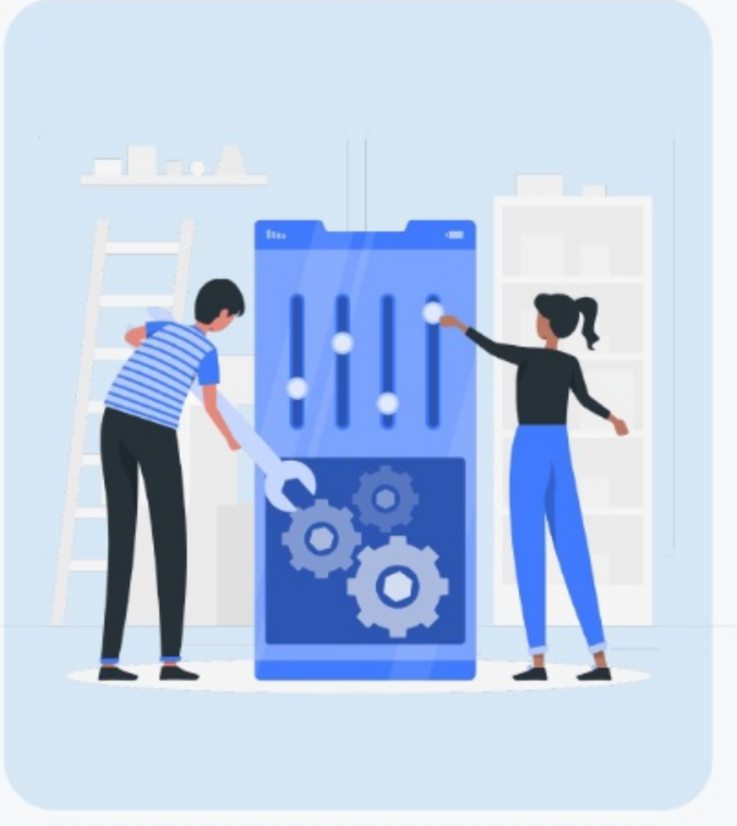
Using Tooling API, **new features and functionality** can be added to existing Lightning Platform tools.

MODULES & TOOLS

Dynamic modules for Lightning Platform development and **specialized development tools** for a specific application or service can be built.

Capabilities of Tooling API

Tooling API can be used for **fine-grained access** to an org's metadata.



RETRIEVING METADATA

Tooling API can be used to retrieve metadata, such as an **object's fields** and its **properties**.

MANAGING METADATA

Tooling API can be used to manage metadata such as **custom fields**, **validation rules**, and **working copies** of Apex classes, triggers, Visualforce pages, components, and static resource files.

APEX CODE & TESTS

Anonymous Apex code can be executed, Apex tests can be **executed**, test **results** can be managed, and **code coverage results** can be accessed.

Tooling API Considerations

Considerations for the use of Tooling API are related to **REST resources** and **SOAP calls**.

REST RESOURCES

Various REST resources are available for querying Tooling API objects. For example, **/query/?q=SOQL_Query_Statement** can be used to execute a query against an object and return data based on the specified criteria.

SOAP CALLS

Tooling API functionality uses SOAP calls such as **query()**, **create()**, **retrieve()**, **delete()**, **search()**, **update()**, **upsert()**, etc. For example, the create() call can be used to compile an Apex class in a sandbox org.

SAVE OPERATIONS WITH WARNINGS

Error-free save operations can be allowed to complete **with warnings** by specifying the header **ignoreSaveWarnings** in the HTTP request.

Tooling API Limitations

Certain **limitations** are applicable to the use of Tooling API.

SOQL OPERATIONS

Some Tooling API objects, such as **EntityDefinition** and **FieldDefinition**, do not support these **SOQL operations**: COUNT(), GROUP BY, LIMIT, LIMIT OFFSET, OR, NOT, and INCLUDES

SOSL OPERATIONS

EntityDefinition and **FieldDefinition** do not support certain **SOSL operations**, such as using multiple wildcards in a search term and using the OR or NOT operators.

CRUD OPERATIONS

CRUD operations in active orgs are **unsupported** for **ApexClass**, **ApexComponent**, **ApexClass**, **ApexPage**, **CustomField**, and **CustomObject**.

Business Scenario

Table of Contents

Business Scenario

Read the business scenario below and determine the most appropriate solutions for the company's requirements.

Cosmic Food Innovations, a prominent leader in the food and beverage industry, is on the brink of revolutionizing supply chain and customer relationship management through enhanced Salesforce capabilities. The enterprise leverages Sales Cloud for tracking customer interactions, Service Cloud for customer service, and custom Salesforce apps tailored to unique business processes in the food sector. With operations spanning across continents and a commitment to delivering top-notch products, the CTO of the company would like to refine the metadata deployment methodologies used by the development team to ensure seamless and efficient updates to applications. Its developers currently use Change Sets for deployments. The company's Salesforce architect is tasked with evaluating the current deployment approaches and identifying the optimal mix of tools and methods to meet varied business needs using Metadata and Tooling APIs.

Requirements & Solutions

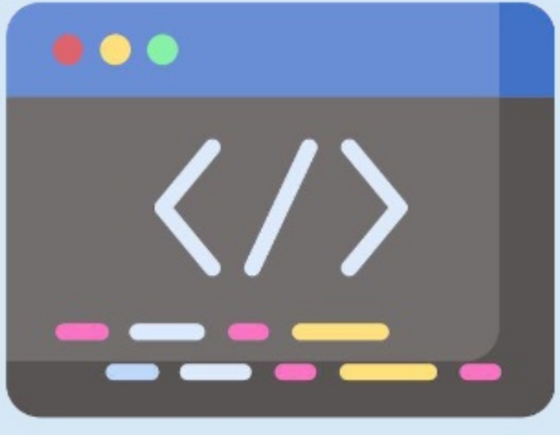
Table of Contents

Requirement 1 (Part 1)

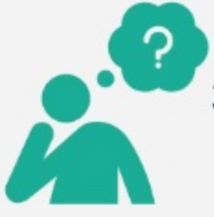
SCENARIO

The Salesforce development team of the company is currently facing the following challenges:

1) The team needs an appropriate deployment strategy before starting to use Salesforce Extensions for VS Code for developing certain custom Lightning web components that will utilize various custom metadata types.
2) The executives are complaining that reports and dashboards roll back to previous versions after each major release. The architect needs to suggest how to stop rollbacks.

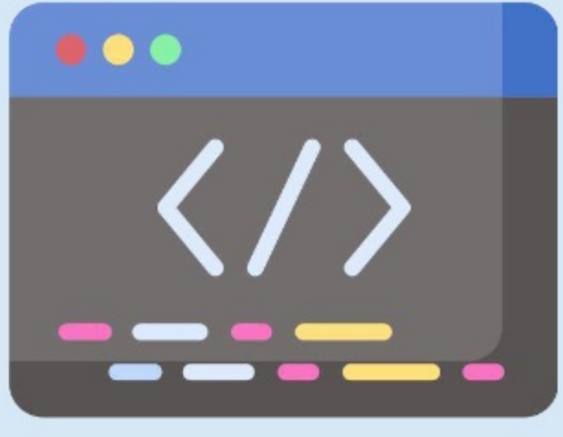


Requirement 1 (Part 2)



SCENARIO

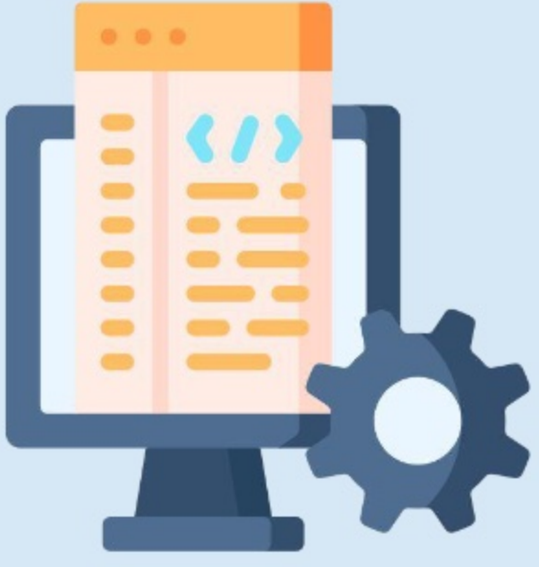
3) Sometimes developers need to remove newly deployed features and would like to automate the process instead of deleting the deployed components manually.
4) A developer has obtained a deployment script that uses API version 58.0. She needs to deploy a new Apex class that uses a standard object that was introduced in API version 59.0. The architect must provide any related considerations.
5) The company wants better control over changes made in the production org and automate them.



Solution 1 (Part 1)

SOLUTION

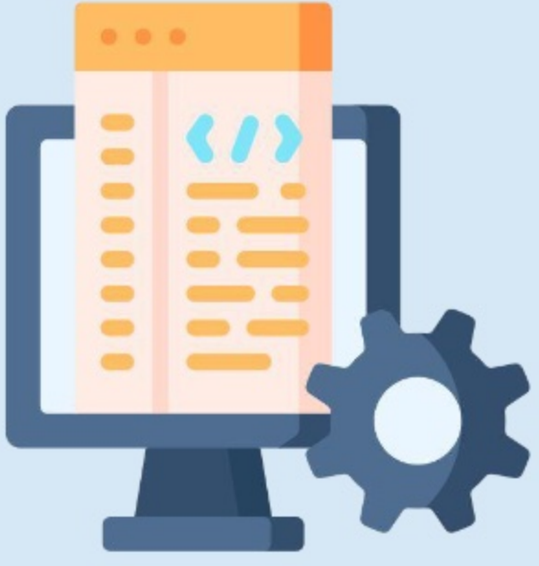
1) The development team can use the **retrieve()** Metadata API call to retrieve existing metadata for development in a scratch org. Once development and testing are complete, the updated metadata can be deployed to a sandbox environment using the **deploy()** call for quality assurance. **CustomObject** and **CustomMetadata** should be included for the deployment of custom metadata types and records. **Salesforce CLI** commands can be utilized for both the calls.
2) To stop rollbacks of reports and dashboards, the metadata should be **exported** daily as **XML files** and **merged** into the development branch before the next release.



Solution 1 (Part 2)

SOLUTION

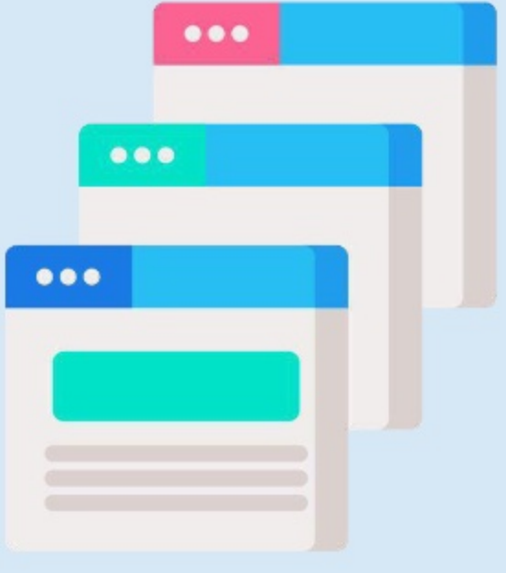
3) To delete the deployed components, a developer can use the **destructiveChanges.xml** file along with **package.xml**. The destructiveChanges.xml file should list the components that should be deleted. The procedure for deleting components is the same as the one used for deploying components.
4) The deployment script would **fail** because it would not recognize the new object introduced in API version 59.0. It would be necessary to use a script that uses API version 59.0 or later.
5) **No changes** should be allowed **directly** in the production org, and **Metadata API** should be used to automate deployments.



Requirement 2

SCENARIO

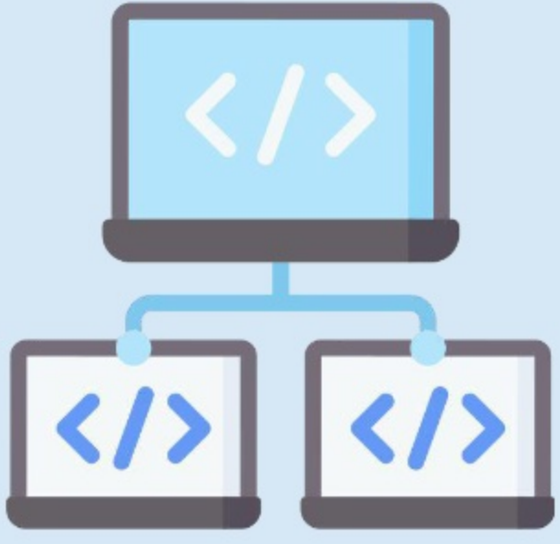
The company recently set up a new production org for its operations in Europe. It would like to deploy various custom applications to this new org. However, this requires deploying a huge volume of metadata. Furthermore, a developer has been tasked with working on an external Java application that allows retrieving and making changes to smaller pieces of metadata, such as custom field definitions. The architect must suggest suitable strategies for these use cases and also provide any limitations of Metadata API that should be considered for deployment to the new production org.



Solution 2

SOLUTION

For the successful deployment of a huge volume of metadata to the new production org, **metadata dependencies** should be identified, **logical groupings** should be created, and the **metadata components** should be **deployed in the right order**.
However, when using Metadata API, it is necessary to consider that only **up to 10,000 files** can be **retrieved or deployed at once** and the **maximum size** of a **deployed .zip file** is **39 MB**.
The external Java application should make use of **Tooling API**, utilizing **SOQL queries** to retrieve and make changes to smaller pieces of Salesforce metadata, such as custom field definitions.
REST resources or SOAP calls can be used for the operations.



Learn More

- Understanding Metadata API
- Introducing Tooling API
- Get to Know the Salesforce Platform APIs